

LAB 4 – VIBECODING CHAT HISTORY

Name: Poojitha V
SRN: PES2UG24AM113
Project: Missile Command

Task 1 – Fix Battery Selection

Prompt: Fix the battery selection so that a destroyed or depleted battery is not selected.

The `nearest_battery()` function was modified to select only batteries that are alive and have ammunition greater than 0. If no usable battery is available, it returns `None`.

```
def nearest_battery(self, target):  
    available = [b for b in self.batteries if b.alive and b.ammo > 0]  
    if not available:  
        return None  
    return min(available, key=lambda b: b.pos.distance_squared_to(target))
```

The `launch()` function was also modified so that an interceptor is launched only when a usable battery is available.

```
def launch(self, target):  
    target = pygame.Vector2(target)  
    if self.state != "play" or target.y > GROUND_Y - 20:  
        return  
    battery = self.nearest_battery(target)  
    if battery is not None:  
        battery.ammo -= 1  
        self.interceptors.append(Interceptor(battery.pos, target))
```

The game was run successfully after making the changes.

Task 2 – Implement explosion_color()

Prompt: Implement the `explosion_color(progress)` function.

The function was implemented so that the explosion color changes according to its progress from 0 to 1.

```
def explosion_color(progress):  
    progress = max(0.0, min(1.0, progress))  
    r = 255  
    g = int(220 * (1 - progress))  
    b = int(60 * (1 - progress))  
    return (r, g, b)
```

The game was executed successfully after implementing the explosion color function.

Task 3 – Implement on_city_destroyed(city)

Prompt: Implement the `on_city_destroyed(city)` function.

The function is called when a missile destroys a city that was still standing. A warning message was added whenever a city is destroyed.

```
def on_city_destroyed(city):  
    print(f"Warning: City at {int(city.pos.x)} has been destroyed!")
```

Testing output:

```
Warning: City at 570 has been destroyed!  
Warning: City at 650 has been destroyed!  
Warning: City at 150 has been destroyed!
```

Task 4 – Implement city_repair_threshold()

Prompt: Implement the `city_repair_threshold()` function.

The function should return an integer score value or `None` to disable city repair. The threshold was set to 2000 as suggested in the README.

```
def city_repair_threshold():  
    return 2000
```

The existing game logic automatically rebuilds one destroyed city when the score crosses a new multiple of the threshold.

Final Testing

The completed game was executed using `python game.py`. The game opened successfully using Pygame. City-destruction warning messages appeared in the Command Prompt, confirming that the implemented city-destruction callback was working. The final game was recorded for the After Video.

GitHub Work

The modified game.py was staged, committed, and pushed to the personal GitHub repository.

```
git add game.py  
git commit -m "Implement remaining game tasks"  
git push origin main
```

Final commit: 2751c85 – Implement remaining game tasks

Repository: poojithav77/33_missileCommand